

## 4.4 Simple Justification Techniques

Sometimes, we will want to make claims about an algorithm, such as showing that it is correct or that it runs fast. In order to rigorously make such claims, we must use mathematical language, and in order to back up such claims, we must justify or *prove* our statements. Fortunately, there are several simple ways to do this.

### 4.4.1 By Example

Some claims are of the generic form, “There is an element  $x$  in a set  $S$  that has property  $P$ .” To justify such a claim, we only need to produce a particular  $x$  in  $S$  that has property  $P$ . Likewise, some hard-to-believe claims are of the generic form, “Every element  $x$  in a set  $S$  has property  $P$ .” To justify that such a claim is false, we only need to produce a particular  $x$  from  $S$  that does not have property  $P$ . Such an instance is called a *counterexample*.

**Example 4.17:** Professor Amongus claims that every number of the form  $2^i - 1$  is a prime, when  $i$  is an integer greater than 1. Professor Amongus is wrong.

**Justification:** To prove Professor Amongus is wrong, we find a counterexample. Fortunately, we need not look too far, for  $2^4 - 1 = 15 = 3 \cdot 5$ . ■

### 4.4.2 The “Contra” Attack

Another set of justification techniques involves the use of the negative. The two primary such methods are the use of the *contrapositive* and the *contradiction*. To justify the statement “if  $p$  is true, then  $q$  is true,” we establish that “if  $q$  is not true, then  $p$  is not true” instead. Logically, these two statements are the same, but the latter, which is called the *contrapositive* of the first, may be easier to think about.

**Example 4.18:** Let  $a$  and  $b$  be integers. If  $ab$  is even, then  $a$  is even or  $b$  is even.

**Justification:** To justify this claim, consider the contrapositive, “If  $a$  is odd and  $b$  is odd, then  $ab$  is odd.” So, suppose  $a = 2j + 1$  and  $b = 2k + 1$ , for some integers  $j$  and  $k$ . Then  $ab = 4jk + 2j + 2k + 1 = 2(2jk + j + k) + 1$ ; hence,  $ab$  is odd. ■

Besides showing a use of the contrapositive justification technique, the previous example also contains an application of *de Morgan’s law*. This law helps us deal with negations, for it states that the negation of a statement of the form “ $p$  or  $q$ ” is “not  $p$  and not  $q$ .” Likewise, it states that the negation of a statement of the form “ $p$  and  $q$ ” is “not  $p$  or not  $q$ .”

### Contradiction

Another negative justification technique is justification by **contradiction**, which also often involves using de Morgan's law. In applying the justification by contradiction technique, we establish that a statement  $q$  is true by first supposing that  $q$  is false and then showing that this assumption leads to a contradiction (such as  $2 \neq 2$  or  $1 > 3$ ). By reaching such a contradiction, we show that no consistent situation exists with  $q$  being false, so  $q$  must be true. Of course, in order to reach this conclusion, we must be sure our situation is consistent before we assume  $q$  is false.

**Example 4.19:** Let  $a$  and  $b$  be integers. If  $ab$  is odd, then  $a$  is odd and  $b$  is odd.

**Justification:** Let  $ab$  be odd. We wish to show that  $a$  is odd and  $b$  is odd. So, with the hope of leading to a contradiction, let us assume the opposite, namely, suppose  $a$  is even or  $b$  is even. In fact, without loss of generality, we can assume that  $a$  is even (since the case for  $b$  is symmetric). Then  $a = 2j$  for some integer  $j$ . Hence,  $ab = (2j)b = 2(jb)$ , that is,  $ab$  is even. But this is a contradiction:  $ab$  cannot simultaneously be odd and even. Therefore,  $a$  is odd and  $b$  is odd. ■

---

### 4.4.3 Induction and Loop Invariants

Most of the claims we make about a running time or a space bound involve an integer parameter  $n$  (usually denoting an intuitive notion of the “size” of the problem). Moreover, most of these claims are equivalent to saying some statement  $q(n)$  is true “for all  $n \geq 1$ .” Since this is making a claim about an infinite set of numbers, we cannot justify this exhaustively in a direct fashion.

#### Induction

We can often justify claims such as those above as true, however, by using the technique of **induction**. This technique amounts to showing that, for any particular  $n \geq 1$ , there is a finite sequence of implications that starts with something known to be true and ultimately leads to showing that  $q(n)$  is true. Specifically, we begin a justification by induction by showing that  $q(n)$  is true for  $n = 1$  (and possibly some other values  $n = 2, 3, \dots, k$ , for some constant  $k$ ). Then we justify that the inductive “step” is true for  $n > k$ , namely, we show “if  $q(j)$  is true for all  $j < n$ , then  $q(n)$  is true.” The combination of these two pieces completes the justification by induction.

**Proposition 4.20:** Consider the Fibonacci function  $F(n)$ , which is defined such that  $F(1) = 1$ ,  $F(2) = 2$ , and  $F(n) = F(n-2) + F(n-1)$  for  $n > 2$ . (See Section 2.2.3.) We claim that  $F(n) < 2^n$ .

**Justification:** We will show our claim is correct by induction.

**Base cases:** ( $n \leq 2$ ).  $F(1) = 1 < 2 = 2^1$  and  $F(2) = 2 < 4 = 2^2$ .

**Induction step:** ( $n > 2$ ). Suppose our claim is true for all  $j < n$ . Since both  $n-2$  and  $n-1$  are less than  $n$ , we can apply the inductive assumption (sometimes called the “inductive hypothesis”) to imply that

$$F(n) = F(n-2) + F(n-1) < 2^{n-2} + 2^{n-1}.$$

Since

$$2^{n-2} + 2^{n-1} < 2^{n-1} + 2^{n-1} = 2 \cdot 2^{n-1} = 2^n,$$

we have that  $F(n) < 2^n$ , thus showing the inductive hypothesis for  $n$ . ■

Let us do another inductive argument, this time for a fact we have seen before.

**Proposition 4.21:** (which is the same as Proposition 4.3)

$$\sum_{i=1}^n i = \frac{n(n+1)}{2}.$$

**Justification:** We will justify this equality by induction.

**Base case:**  $n = 1$ . Trivial, for  $1 = n(n+1)/2$ , if  $n = 1$ .

**Induction step:**  $n \geq 2$ . Assume the inductive hypothesis is true for any  $j < n$ . Therefore, for  $j = n-1$ , we have

$$\sum_{i=1}^{n-1} i = \frac{(n-1)(n-1+1)}{2} = \frac{(n-1)n}{2}.$$

Hence, we obtain

$$\sum_{i=1}^n i = n + \sum_{i=1}^{n-1} i = n + \frac{(n-1)n}{2} = \frac{2n + n^2 - n}{2} = \frac{n^2 + n}{2} = \frac{n(n+1)}{2},$$

thereby proving the inductive hypothesis for  $n$ . ■

We may sometimes feel overwhelmed by the task of justifying something true for *all*  $n \geq 1$ . We should remember, however, the concreteness of the inductive technique. It shows that, for any particular  $n$ , there is a finite step-by-step sequence of implications that starts with something true and leads to the truth about  $n$ . In short, the inductive argument is a template for building a sequence of direct justifications.

### Loop Invariants

The final justification technique we discuss in this section is the *loop invariant*. To prove some statement  $\mathcal{L}$  about a loop is correct, define  $\mathcal{L}$  in terms of a series of smaller statements  $\mathcal{L}_0, \mathcal{L}_1, \dots, \mathcal{L}_k$ , where:

1. The *initial* claim,  $\mathcal{L}_0$ , is true before the loop begins.
2. If  $\mathcal{L}_{j-1}$  is true before iteration  $j$ , then  $\mathcal{L}_j$  will be true after iteration  $j$ .
3. The final statement,  $\mathcal{L}_k$ , implies the desired statement  $\mathcal{L}$  to be true.

Let us give a simple example of using a loop-invariant argument to justify the correctness of an algorithm. In particular, we use a loop invariant to justify that the method `arrayFind` (see Code Fragment 4.11) finds the smallest index at which element `val` occurs in array `A`.

```

1  /** Returns index j such that data[j] == val, or -1 if no such element. */
2  public static int arrayFind(int[] data, int val) {
3      int n = data.length;
4      int j = 0;
5      while (j < n) { // val is not equal to any of the first j elements of data
6          if (data[j] == val)
7              return j; // a match was found at index j
8              j++; // continue to next index
9          // val is not equal to any of the first j elements of data
10     }
11     return -1; // if we reach this, no match found
12 }
```

**Code Fragment 4.11:** Algorithm `arrayFind` for finding the first index at which a given element occurs in an array.

To show that `arrayFind` is correct, we inductively define a series of statements,  $\mathcal{L}_j$ , that lead to the correctness of our algorithm. Specifically, we claim the following is true at the beginning of iteration  $j$  of the **while** loop:

$\mathcal{L}_j$ : *val* is not equal to any of the first  $j$  elements of data.

This claim is true at the beginning of the first iteration of the loop, because  $j$  is 0 and there are no elements among the first 0 in data (this kind of a trivially true claim is said to hold *vacuously*). In iteration  $j$ , we compare element *val* to element `data[j]`; if these two elements are equivalent, we return the index  $j$ , which is clearly correct since no earlier elements equal *val*. If the two elements *val* and `data[j]` are not equal, then we have found one more element not equal to *val* and we increment the index  $j$ . Thus, the claim  $\mathcal{L}_j$  will be true for this new value of  $j$ ; hence, it is true at the beginning of the next iteration. If the while loop terminates without ever returning an index in data, then we have  $j = n$ . That is,  $\mathcal{L}_n$  is true—there are no elements of data equal to *val*. Therefore, the algorithm correctly returns `-1` to indicate that *val* is not in data.